PYTHON CHEATSHEET

1. PYTHON BASICS

1.1 What is Python ?

- Python is a high-level, interpreted, general-purpose programming language.
- It was created by Guido van Rossum and first released in 1991.
- Python is easy to learn and has simple syntax similar to English.
- It is widely used in web development, data science, AI, automation, scripting and more.

Key Features

* Easy to learn and use
* Readable and clean syntax
* Interpreted Language
* Cross-platform
* Large standard library
* Supports multiple paradigms (OOP, Procedural, Functional)

1.2 First Python Program

```
# This is a simple Python program
print ("Hello, World!")
```

→

Output:

```
Hello, World!
```

1.3 Comments in Python

- Comments are used to explain the code. Python supports single-line and multi-line comments.

Single-line comment

```
# This is a single-line comment
print ("Hello")  # This prints Hello
```

Multi-line comment

```
"""
This is a
multi-line comment.
It can span across
multiple lines.
"""
```

Note:

- Comments are ignored by the Python interpreter.
- Use comments to make your code more readable and understandable.

PYTHON CHEATSHEET

2. VARIABLES & DATA TYPES

2.1 Variables

- A variable is a container that stores a value.
- In Python, we don't need to declare the type of variable. It is inferred automatically.
- Variable names are case-sensitive.

Rules for Variable Names

* Must start with a letter (a-z, A-Z) or underscore (_).
* Can contain letters, digits (0-9) and underscore (_).
* Cannot start with a digit.
* Cannot be a keyword.

2.2 Data Types

Data Type	Description	Example
int	Integer numbers	x = 10
float	Decimal numbers	y = 3.14
str	Sequence of characters	name = "CodeNotes"
bool	Boolean values (True / False)	flag = True
list	Ordered collection (mutable)	nums = [1, 2, 3]
tuple	Ordered collection (immutable)	t = (1, 2, 3)
set	Unordered collection of unique items	s = {1, 2, 3}
dict	Key-value pairs	d = {"a": 1, "b": 2}

2.3 type() Function

- type() function is used to check the data type of a variable.

Examples:

```
type(10)          # <class 'int'>
type(3.14)        # <class 'float'>
type("Hello")     # <class 'str'>
type(True)        # <class 'bool'>
type([1,2,3])     # <class 'list'>
type((1,2,3))     # <class 'tuple'>
type({1,2,3})     # <class 'set'>
type({"a": 1})    # <class 'dict'>
```

2.4 Examples

```
a = 25                          # int
b = 3.75                        # float
name = "Alice"                  # str
is_active = True                # bool
numbers = [1, 2, 3, 4]          # list
coordinates = (10.0, 20.0)      # tuple
unique = {1, 2, 3, 4}           # set
info = {"name": "Alice", "age": 20} # dict
```

Quick Tips

* Variable names should be meaningful.
* Use snake_case for variable names (e.g., user_name).
* Python keywords cannot be used as variable names.
* Use type() to debug and understand your data.

☆ Variables store data. Data types define the kind of data. ☆

PYTHON CHEATSHEET

5. CONDITIONAL STATEMENTS

Conditional statements are used to make decisions in a program.

5.1 if Statement

- Executes a block of code if the condition is True.

Syntax:

```
if condition:
    # code to be executed
```

Example:

```
x = 10
if x > 5:
    print ("x is greater than 5")
```

5.2 if-else Statement

- Executes one block if the condition is True otherwise executes another block.

Syntax:

```
if condition:
    # code if True
else:
    # code if False
```

Example:

```
x = 3
if x % 2 == 0:
    print ("Even number")
else:
    print ("Odd number")
```

5.3 if-elif-else Statement

- Checks multiple conditions.

Syntax:

```
if condition1:
    # code if condition1 is True
elif condition2:
    # code if condition2 is True
else:
    # code if all conditions are False
```

Example:

```
marks = 75
if marks >= 90:
    print ("Grade: A")
elif marks >= 75:
    print ("Grade: B")
elif marks >= 50:
    print ("Grade: C")
else:
    print ("Grade: D")
```

5.4 Nested if Statement

- An if statement inside another if or else block.

Example:

```
x = 20
if x > 10:
    if x % 2 == 0:
        print ("x is even and > 10")
    else:
        print ("x is odd and > 10")
else:
    print ("x is <= 10")
```

Quick Tips

* Indentation is very important.
* Python uses indentation to define blocks.
* Use meaningful conditions.
* You can have multiple elif blocks.

Remember: The program flows from top to bottom. Conditions help the program choose the right path

PYTHON CHEATSHEET

4. OPERATORS

Operators are used to perform operations on variables and values.

4.1 Arithmetic Operators

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 3	1.6667
//	Floor Division	5 // 3	1
%	Modulus	5 % 3	2
**	Exponentiation	5 ** 3	125

Note: // gives the integer part (floored value).
% gives the remainder.

4.2 Comparison (Relational) Operators

Operator	Name	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 3	True
<=	Less than or equal to	5 <= 3	False

Note: Comparison operators always return Boolean values (True or False).

4.3 Logical Operators

Operator	Name	Example	Result
and	Logical AND	(5 > 3) and (2 < 4)	True
or	Logical OR	(5 > 3) or (2 > 4)	True
not	Logical NOT	not (5 > 3)	False

Note: and - True if both are True
or - True if at least one is True
not - Reverses the result

4.4 Assignment Operators

Operator	Example	Same as
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

4.5 Membership Operators

Operator	Name	Example	Result
in	Returns True if value is found in sequence	'a' in 'apple'	True
not in	Returns True if value is not found in sequence	'z' not in 'apple'	True

Works with strings, lists, tuples, sets, dictionaries.

4.6 Identity Operators

Operator	Name	Example	Result
is	True if both variables refer to the same object	a is b	True/False
is not	True if both variables do not refer to the same object	a is not b	True/False

Checks memory location, not just value.

Quick Tip: Use parentheses to make expressions clear.

PYTHON CHEATSHEET

3. INPUT & OUTPUT

3.1 Input in Python

- input() function is used to take input from the user.
- By default, input() returns data as a string.

Syntax:

```
input(prompt)
```

Example:

```
name = input("Enter your name: ")
print("Hello", name)
```

Type Conversion

If you want integer or float input, convert it using int() or float().

Example:

```
age = int(input("Enter your age:"))
height = float(input("Enter height:"))
```

3.2 Output in Python

- print() function is used to display output on the screen.
- It can print text, numbers, or variables.

3.3 print() Syntax

```
print(value1, value2, ..., sep=' ', end='\n')
```

- sep → separates values (default is space)
- end → what to print at the end (default is new line)

Example:

```
print("Hello", "Python")     # Hello Python
print(10, 20, sep="-")       # 10-20
print("Hi", end="! ")        # Hi!
print("Welcome")             # Welcome
```

3.4 String Formatting (f-strings)

- f-strings provide a simple way to include variables inside strings.
- Start the string with 'f' or 'F' and use { } to insert variables.

Example:

```
name = "Alice"
age = 20
print(f"My name is {name} and
      I am {age} years old.")
```

Other Formatting Examples:

```
a, b = 5, 10
print(f"Sum: {a + b}")          # Sum: 15
price = 99.99
print(f"Price: ₹{price:.2f}")   # Price: ₹99.99
```

☆ **Quick Tip:** Use input() to get data from the user and print() ☆

PYTHON CHEATSHEET

8. DATA STRUCTURES

Data structures are used to store, organize and manage data efficiently.

8.1 List (Mutable)

- Ordered collection of items.
- Items can be of different data types.
- Lists are mutable (can be changed).

Syntax:

```
my_list = [item1, item2, item3]
```

Example:

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")       # add item
fruits[1] = "blueberry"       # change item
print(fruits)             # ['apple', 'blueberry',
                                'mango', 'orange']
```

Common Operations:

Operation	Example
Access	fruits[0] # 'apple'
Append	fruits.append("orange")
Insert	fruits.insert(1, "grape")
Remove	frints.remove("banana")
Length	len(fruits)

8.2 Tuple (Immutable)

- Ordered collection of items.
- Items can be of different data types.
- Tuples are immutable (cannot be changed).

Syntax:

```
my_tuple = (item1, item2, item3)
```

Example:

```
numbers = (1, 2, 3, 4, 5)
print(numbers[0])          # 1
print(numbers.index(3))    # 2
print(len(numbers))        # 5
```

Note: Tuples don't have append(), remove(), etc.

Common Operations:

Operation	Example
Access	numbers[2] # 3
Index	numbers.index(4) # 3
Count	numbers.count(2) # 1
Length	len(numbers)

8.3 Set (Unordered, Unique)

- Unordered collection of unique items.
- No duplicate values allowed.
- Useful for membership testing and removing duplicates.

Syntax:

```
my_set = {item1, item2, item3}
```

Example:

```
my_set = {1, 2, 3, 2, 4}    # duplicates removed
my_set.add(5)               # {1, 2, 3, 4, 5}
my_set.remove(2)            # {1, 3, 4, 5}
print(3 in my_set)          # True
```

Common Operations:

Operation	Example
Add	my_set.add(10)
Remove	my_set.remove(10)
Union	set1.union(set2)
Intersection	set1.intersection(set2)

8.4 Dictionary (Key-Value Pair)

- Stores data in key-value pairs.
- Keys must be unique and immutable.
- Values can be of any data type.

Syntax:

```
my_dict = {key1: value1, key2: value2}
```

Example:

```
student = {"name": "Alice", "age": 20, "grade": "A"}
print(student["name"])       # Alice
student["age"] = 21          # update value
student["city"] = "Delhi"    # add new key-value
print(student.keys())        # dict_keys(['name', 'age',
                                 'grade', 'city'])
```

Common Operations:

Operation	Example
Access	student["name"]
Add / Update	student["age"] = 21
Delete	del student["grade"]
Keys	student.keys()
Values	student.values()
Items	student.items()

Quick Tips

- Use List when you need ordered and changeable data.
- Use Tuple when data should not change.
- Use Set to store unique items.
- Use Dictionary when you need key-value mapping.

Comparison

Structure	Ordered	Mutable	Allows Duplicates	Access by Index	Key-Value Pairs
List	Yes	Yes	Yes	Yes	No
Tuple	Yes	No	Yes	Yes	No
Set	No	Yes	No	No	No
Dictionary	No*	Yes	No (keys)	No	Yes

PYTHON CHEATSHEET

7. FUNCTIONS

Functions are used to group a block of code together to perform a specific task.

7.1 Defining a Function

- A function is defined using the def keyword.
- It may or may not return a value.

Syntax:

```
def function_name(parameters):
    # code block
    return value  # optional
```

Example:

```
def greet():
    print("Hello, Welcome!")

greet()   # function call
```

7.2 Function with Parameters

- Parameters are variables listed in the function definition.
- Arguments are the values passed during call.

Syntax:

```
def function_name(parameters):
    # code block
    return value  # optional
```

Example:

```
def add(a, b):
    result = a + b
    return result

sum = add(5, 3)
print(sum)    # 8
```

7.3 Function with Return Statement

- The return statement sends a value back to the caller.
- The function stops executing after return.

Example:

```
def square(n):
    return n * n
x = square(4)
print(x)   # 16
```

7.4 Default Parameters

- If an argument is not passed, the default value is used.

Example:

```
def greet(name = "Guest"):
    print("Hello, " + name)

greet()              # Hello, Guest
greet("Alice")       # Hello, Alice
```

7.5 Variable Length Arguments

- *args is used to pass non-keyworded, variable length arguments.

Example:

```
def total(*numbers):
    s = 0
    for n in numbers:
        s = s + n
    return s
print(total(1, 2, 3))          # 6
print(total(5, 10, 15, 20))    # 50
```

7.6 Recursive Function

- A function that calls itself.

Example:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)

print(factorial(5))     # 120
```

Advantages of Functions

- ★ Reduces code duplication.
- ★ Improves readability.
- ★ Makes code easy to test and debug.
- ★ Supports modular programming.

Quick Tips

- * Use meaningful function names.
- * Functions can return only one value.
- * Docstrings are a good practice for documentation.
- * Keep functions small and focused on one task.

PYTHON CHEATSHEET

6. LOOPS

Loops are used to repeat a block of code multiple times.

6.1 for Loop

- Used to iterate over a sequence (string, list, tuple, range, etc.)
- Executes the block once for each item.

Syntax:

```
for variable in sequence:
    # code to be executed
```

Example:

```
fruits = ["apple", "banana", "mango"]
for fruit in fruits:
    print(fruit)
# Output: apple, banana, mango
```

6.2 while Loop

- Repeats a block of code while the condition is True.
- The block may execute zero or more times.

Syntax:

```
while condition:
    # code to be executed
```

Example:

```
i = 1
while i <= 5:
    print(i)
    i += 1
# Output: 1 2 3 4 5
```

6.3 range() Function

- Generates a sequence of numbers.
- range(start, stop, step) → stop is exclusive.

Syntax	Description
range(stop)	0 to stop-1
range(start, stop)	start to stop-1
range(start, stop, step)	start to stop-1 with step

Example:

```
range(5)          → 0, 1, 2, 3, 4
range(2, 6)       → 2, 3, 4, 5
range(1, 10, 2)   → 1, 3, 5, 7, 9
```

6.4 Loop Control Statements

Statement	Description	Example

break	Terminates the loop completely	`for i in range(5):`
		`  if i == 3:`
		`  break`

continue	Skips the current iteration and continues with next	`for i in range(5):`
		`  if i == 2:`
		`  continue`
		`print(i)`

pass	Does nothing. Used as a placeholder.	`for i in range(5):`
		`  if i == 2:`
		`  pass`
		`print(i)`

6.5 Nested Loops

- A loop inside another loop.
- Useful for working with matrices, patterns, etc.

Example:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} {j}", end="  ")
    print()   # new line after each row
```

Output:

```
1 1   1 2   1 3
2 1   2 2   2 3
3 1   3 2   3 3
```

Quick Tips

* Use for loop when you know the number of iterations.
* Use while loop when the number of iterations is unknown.
* Use break, continue and pass to control the flow of loops.

Loops make your code shorter, smarter and easier to manage.

11. EXCEPTION HANDLING

Exception handling helps us handle errors gracefully without crashing the program.

11.1 What is an Exception?

- An exception is an error that occurs during execution.
- It disrupts the normal flow of the program.

Example (Without Exception Handling):

```
a = 10
b = 0
c = a / b        # ZeroDivisionError
print(c)         # Program crashes here
```

11.2 try and except

- The try block contains code that may raise an exception.
- The except block handles the exception.

Syntax:

```
try:
    # risky code
except ExceptionType:
    # handle the exception
```

Example:

```
try:
    a = 10                              # risky code
    b = 0
    c = a / b
except ZeroDivisionError:
    print("Cannot divide by zero!")     # handle error
print("Program continues...")           # rest of code
```

11.3 try, except, else

- else block runs if no exception occurs in try block.
- Useful for code that should run only when try succeeds.

Syntax:

```
try:
    # risky code
except ExceptionType:
    # handle exception
else:
    # runs if no exception
```

Example:

```
try:
    num = int(input("Enter a number: "))
    print("You entered:", num)
except ValueError:
    print("Invalid input! Please enter a number.")
else:
    print("Input is valid. Thank you!")
```

11.4 try, except, else, finally

- finally block always runs, whether exception occurs or not.
- Used for cleanup activities (like closing files).

Syntax:

```
try:
    # risky code
except ExceptionType:
    # handle exception
else:
    # runs if no exception
finally:
    # always runs
```

Example:

```
try:
    f = open("example.txt", "r")          # risky code
    data = f.read()
except FileNotFoundError:
    print("File not found!")              # handle error
else:
    print("File read successfully")       # no error
finally:
    f.close()    # this runs always       # cleanup code
    print("File closed")
```

11.5 Raising Exceptions

- We can raise exceptions manually using raise keyword.
- Useful for custom validations.

Syntax:

```
raise ExceptionType("Error message")
```

Example:

```
age = -5
if age < 0:                                   # raises exception
    raise ValueError("Age cannot be negative!")
```

11.6 Multiple Except Blocks

- Handle different exceptions with different except blocks.

Example:

```
try:
    x = int(input("Enter a number: "))
    y = 10 / x
except ValueError:
    print("Please enter a valid number.")
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print("Something went wrong:", e)
```

11.7 Common Exceptions in Python

Exception	Description	Example
ZeroDivisionError	Division by zero	10 / 0
ValueError	Invalid value	int("abc")
TypeError	Wrong data type	"5" + 5
FileNotFoundError	File not found	open("file.txt")
IndexError	Index out of range	list[10]
KeyError	Key not found in dict	dict["key"]

Quick Tips

- Use specific exceptions instead of generic Exception.
- Keep try block minimal.
- Always use finally to release resources.
- Exception handling makes your code robust and user-friendly.

. Handle Exceptions, Not your Program! .

PYTHON CHEATSHEET

10. FILE HANDLING

File handling allows us to work with files on disk - read, write, and manage data.

10.1 Opening a File

- open() function is used to open a file.
- Syntax: open(filename, mode)

Mode	Description
'r'	Read (default) - file must exist
'w'	Write - creates new file or truncates existing
'a'	Append - adds data to the end of file
'x'	Create - creates new file, returns error if exists
'b'	Binary mode
't'	Text mode (default)
'+'	Read and write

Example:

```
f = open("example.txt", "r")      # open for reading
f = open("data.txt", "w")         # open for writing
f = open("log.txt", "a")          # open for appending
f = open("image.png", "rb")       # open binary file
f = open("update.txt", "r+")      # read and write
```

10.2 Reading from a File

- Data can be read using different methods.
- File must be opened in 'r' or 'r+' mode.

Method	Description
read()	Reads entire file
read(size)	Reads 'size' characters
readline()	Reads one line
readlines()	Reads all lines and returns list

Example:

```
f = open("example.txt", "r")
text = f.read()              # entire file
line = f.readline()          # one line
lines = f.readlines()        # list of lines
f.close()                    # always close file
```

10.3 Writing to a File

- Data can be written using write() or writelines().
- File must be opened in 'w', 'a' or 'w+" mode.

Example:

```
f = open("output.txt", "w")
f.write("Hello, Python!\n")          # write string
f.write("Learning File Handling.\n")
f.writelines(["Line 1\n", "Line 2\n", "Line 3\n"])
f.close()                            # close file
```

10.4 Appending to a File

- New data is added at the end without deleting existing data.
- Use 'a' or 'a+" mode.

Example:

```
f = open("log.txt", "a")
f.write("New entry 1\n")
f.write("New entry 2\n")
f.close()
```

10.5 Closing a File

- close() is used to close the file.
- It is important to close the file after use.

Example:

```
f = open("example.txt", "r")
# do some operations
f.close()    # file is now closed
```

10.6 Using with Statement (Best Practice)

- Automatically handles closing of the file.
- Recommended to use with statement.

Example:

```
with open("example.txt", "r") as f:
    data = f.read()
    print(data)
# file is closed automatically
```

10.7 File Pointer Methods

Method	Description
tell()	Returns current position of file pointer
seek(offset)	Moves file pointer to given position
seek(offset, whence)	Moves pointer: 0 - start, 1 - current position, 2 - end

Example:

```
f = open("example.txt", "r")
print(f.tell())      # current position
f.seek(10)           # move to 10th position
data = f.read(5)     # read 5 characters
f.close()
```

10.8 Common File Operations

Operation	Example
Check if file exists	import os os.path.exists("file.txt")
Get file size	os.path.getsize("file.txt")
Remove file	os.remove("file.txt")
Rename file	os.rename("old.txt", "new.txt")

Quick Tips

* Always close the file after use.
* Use with statement to avoid forgetting to close.
* Use appropriate mode: r, w, a, x, r+, w+, a+.
* Handle exceptions using try-except while working with files.
* File handling is case sensitive (file.txt ≠ File.txt).

Good file handling makes your program smart and efficient!

PYTHON CHEATSHEET

9. STRINGS

Strings are sequences of characters enclosed in single, double or triple quotes.

9.1 Creating Strings

- Strings can be created using ' ', " " or """ """.
- Triple quotes are used for multi-line strings.

Example:

```
s1 = 'Hello'                # single quotes
s2 = "Python"               # double quotes
s3 = """This is             # triple quotes
    a multi-line string"""
```

9.2 Accessing Characters

- Indexing starts from 0.
- Negative indexing starts from -1 (end).

Example:

```
s = "Python"
s[0]        # 'P'
s[3]        # 'h'
s[-1]       # 'n'
s[-2]       # 'o'
```

9.3 Slicing Strings

- Extract a part of the string using start:stop:step

Example:

```
s = "HelloWorld"
s[0:5]        # "Hello'
s[5:10]       # 'World'
s[:]          # "HelloWorld"
s[::2]        # "HloWrd'
s[::-1]       # 'dlroWolleH"
```

9.4 String Methods (Common)

Method	Description
lower()	Convert to lowercase
upper()	Convert to uppercase
strip()	Remove leading and trailing spaces
replace(old, new)	Replace old with new
split(sep)	Split string by separator
join(iterable)	Join iterable with string
find(sub)	Return index of first occurrence
count(sub)	Return number of occurrences

Example:

```
s = "  Python Programming  "
s.lower()                   # ' python programming '
s.strip()                   # 'Python Programming'
s.replace("Python", "Code") #  '  Code Programming  '
s.split()                   # ['Python', 'Programming']
```

9.5 String Formatting

- Old style using % operator
- New style using format() method
- f-strings (Python 3.6+)

Example:

```
name = "Alice"
age = 21
# % operator
s1 = "My name is %s and I am %d years old." % (name, age)
# format() method
s2 = "My name is {} and I am {} years old.".format(name, age)
# f-string
s3 = f"My name is {name} and I am {aye} years old."
print(s1)
print(s2)
print(s3)       # Output: all same
```

9.6 String Operators

Operator	Description	Example
+	Concatenation	"Hello" + " World"
*	Repetition	"Ha" * 3 → "HaHaHa"
in	Membership	"Py" in "Python" → True
not in	Non-membership	"Java" not in "Python" → True

9.7 String Functions (Built-in)

Function	Description
len(s)	Length of string
min(s)	Smallest character
max(s)	Largest character
ord(c)	ASCII value of character
chr(n)	Character from ASCII value

Example:

```
s = "Python"
len(s)          # 6
min(s)          # 'P'
max(s)          # 'y'
ord('A')        # 65
chr(97)         # 'a'
```

Quick Tips

* Strings are immutable (cannot be changed).
* Use single or double quotes unless you need multi-line strings.
* Use slicing to extract parts of a string easily.
* f-strings are the most readable and modern way to format strings.

PYTHON CHEATSHEET

12. MODULES IN PYTHON

Modules help us organize our code into reusable files. We can import modules and use their functions, classes and variables in our programs.

12.1 What is a Module?

- A module is a file containing Python code.
- It can define functions, classes and variables.
- The file name is the module name (without .py).

Example:

File name: mymodule.py

```
# This is a module
# (mymodule.py)
def greet(name):
    return f"Hello, {name}!"
PI = 3.14159
```

12.2 Importing a Module

- We use import statement to use a module.
- The module file must be in the same folder or in Python path.

Syntax	Description
import module_name	Import entire module
from module_name import name	Import specific name (function/variable/class)

Example:

```
import mymodule                 # import entire module
mymodule.greet("Alice")         # Hello, Alice!
print(mymodule.PI)              # 3.14159
```

```
from mymodule import greet, PI
greet("Bob")                    # Hello, Bob!
print(PI)                       # 3.14159
```

12.3 Using as Keyword

- We can give a module a short name using as.
- Useful when module name is long.

Example:

```
import mymodule as mm
mm.greet("Charlie")             # Hello, Charlie!
print(mm.PI)                    # 3.14159

from mymodule import greet as gr
gr("David")                     # Hello, David!
```

Quick Tips



* Use modules to keep code organized.
* Import only what you need.
* Use meaningful names for your own modules.
* Built-in modules save a lot of time!

12.4 Built-in Modules

- Python comes with many built-in modules.
- No need to install them.
- Just import and use.

Module	Description
math	Mathematical functions
random	Generate random numbers
datetime	Work with dates and times
os	Interact with operating system
sys	System-specific parameters

Example:

```
import math
print(math.sqrt(16))            # 4.0
import random
print(random.randint(1, 10))    # random number
from datetime import datetime
print(datetime.now())           # current date & time
import os
print(os.getcwd())              # current working dir
```

12.5 Creating Your Own Module

- Write your code in a .py file.
- Save it and import in another program.

Example: (mymodule.py)

```
def add(a, b):
    return a + b
def square(x):
    return x * x
VALUE = 100
```

Example: (using in another file)

```
import mymodule
print(mymodule.add(5, 7))       # 12
print(mymodule.square(4))       # 16
print(mymodule.VALUE)           # 100
```

12.6 Module Search Path

- Python looks for modules in the following order:
  1. The directory containing the script.
  2. Built-in modules.
  3. Python path (list in sys.path).

Summary



- ✓ Modules make code reusable and organized.
- ✓ Use import to access module features.
- ✓ Explore built-in modules to boost productivity.
- ✓ Create your own modules to solve problems better!